

Android TextToSpeech 完整API详解文档

(Java、全版本兼容)

本文档涵盖Android原生 **TextToSpeech (TTS文字转语音)** 所有核心API、高级用法、版本适配、报错解决方案、队列机制、状态监听、引擎切换、音频导出等内容，兼容 Android 5.0 ~ Android 16+，解决开发中常见的引擎未就绪、初始化报错、播报失效、内存泄漏等问题。

一、TTS 核心基础概念

1.1 什么是 TextToSpeech

TextToSpeech 是Android系统原生提供的**文字转语音合成服务**，支持离线播报，无需联网，系统内置引擎，同时兼容第三方TTS引擎（讯飞、百度、华为、小米语音等）。

1.2 核心特性

- 异步初始化机制：TTS引擎连接为异步操作，未初始化完成调用API会直接报错
- 支持自定义语速、音调、语种
- 支持语音队列管理（追加/清空即时播报）
- 支持监听播报开始、完成、异常状态
- 支持切换系统/第三方TTS引擎
- 支持文字转离线音频文件保存

二、权限配置（必备）

适配全版本TTS功能，需在 AndroidManifest.xml 配置基础权限，音频导出、联网语音合成需额外权限：

代码块

```
1  <!-- 网络语音合成权限 -->
2  <uses-permission android:name="android.permission.INTERNET" />
3  <!-- 音频文件保存权限 (Android10以下) -->
4  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

三、TTS 核心API全解

3.1 初始化 API

TTS 构造方法为异步初始化，必须通过 **OnInitListener** 监听初始化结果，初始化成功后才可调用所有功能API。

代码块

```
1 // 基础构造方法（使用系统默认引擎）
2 TextToSpeech(Context context, OnInitListener listener)
3
4 // 高级构造方法（指定第三方TTS引擎）
5 TextToSpeech(Context context, OnInitListener listener, String
  enginePackageName)
```

初始化状态码：

- `TextToSpeech.SUCCESS`：引擎连接初始化成功
- `TextToSpeech.ERROR`：引擎连接失败、设备无可用语音引擎

核心避坑：杜绝初始化未完成时调用 `setLanguage()`、`speak()`，会抛出 `setLanguage failed: TTS engine connection not fully set up` 错误。

3.2 语种设置 API

代码块

```
1 // 设置播报语种
2 int setLanguage(Locale locale)
```

返回值状态说明：

- `TextToSpeech.LANG_AVAILABLE`：语种可用，设置成功
- `TextToSpeech.LANG_MISSING_DATA`：缺失语音数据包（需用户下载系统语音包）
- `TextToSpeech.LANG_NOT_SUPPORTED`：当前设备引擎不支持该语种

常用语种：`Locale.CHINA`（中文）、`Locale.ENGLISH`（英文）

3.3 语速 & 音调 API

代码块

```
1 // 设置播报语速
2 void setSpeechRate(float rate)
3
4 // 设置播报音调
```

```
5 void setPitch(float pitch)
```

参数取值范围：

- 语速 rate：0.0 ~ 2.0，默认 1.0（数值越大语速越快）
- 音调 pitch：0.0 ~ 2.0，默认 1.0（数值越高音色越尖）

常用适配：慢速0.6f、快速1.5f、萝莉音1.3f、大叔音0.8f

3.4 核心播报 API（队列机制）

TTS 内置语音队列管理，两种播报模式适配不同业务场景，全版本兼容适配。

代码块

```
1 // 通用播报方法（适配全Android版本）
2 int speak(CharSequence text, int queueMode, Bundle params, String utteranceId)
```

队列模式参数：

- `TextToSpeech.QUEUE_ADD`：队列追加模式，保留当前播报，新文本加入队列等待播放
- `TextToSpeech.QUEUE_FLUSH`：清空队列模式，立即终止当前播报，清空所有队列，直接播放新文本

3.5 播报状态监听 API

通过 `UtteranceProgressListener` 精准监听单条语音播报全生命周期，支持逐条语音状态回调。

代码块

```
1 // 设置播报进度全局监听
2 void setOnUtteranceProgressListener(UtteranceProgressListener listener)
```

监听回调方法：

- `onStart(String utteranceId)`：单条语音开始播报
- `onDone(String utteranceId)`：单条语音播报完成
- `onError(String utteranceId)`：播报异常失败

3.6 状态判断 & 控制 API

代码块

```
1 // 判断当前是否正在播报
```

```
2  boolean isSpeaking()  
3  
4  // 停止所有播报、清空队列  
5  void stop()
```

3.7 TTS引擎管理 API

代码块

```
1  // 获取设备所有已安装TTS引擎信息  
2  List<EngineInfo> getEngines()  
3  
4  // 切换指定TTS引擎（需重启初始化）  
5  TextToSpeech(Context context, OnInitListener listener, String  
    enginePackageName)
```

支持适配：系统自带引擎、讯飞语音、百度语音、华为/小米原生TTS引擎

3.8 离线音频导出 API

无需播放，直接将文字合成语音保存为本地音频文件（Android 5.0+ 支持）。

代码块

```
1  int synthesizeToFile(CharSequence text, Bundle params, File file, String  
    utteranceId)
```

3.9 资源释放 API

代码块

```
1  // 关闭播报、销毁引擎连接  
2  void shutdown()
```

必做操作：页面/组件销毁时必须调用，否则会造成TTS服务常驻、内存泄漏。

四、全版本兼容高级工具类（Java）

封装所有核心API，修复版本兼容问题、初始化报错、内存泄漏，可直接商用。

代码块

```
1  import android.content.Context;  
2  import android.os.Build;
```

```

3  import android.speech.tts.TextToSpeech;
4  import android.speech.tts.UtteranceProgressListener;
5  import android.util.Log;
6
7  import java.util.HashMap;
8  import java.util.Locale;
9  import java.util.Map;
10 import java.util.UUID;
11
12 public class AdvancedTTS {
13     private static final String TAG = "AdvancedTTS";
14     private final Context mContext;
15     private TextToSpeech mTts;
16     private boolean isInitSuccess = false;
17
18     // 全生命周期播报监听回调
19     public interface TTSListener {
20         void onTtsInitSuccess();
21         void onTtsInitFailed();
22         void onSpeakStart(String id);
23         void onSpeakDone(String id);
24         void onSpeakError(String id);
25     }
26
27     private TTSListener mListener;
28
29     // 播报参数默认值
30     private float mSpeechRate = 1.0f;
31     private float mPitch = 1.0f;
32
33     public AdvancedTTS(Context context) {
34         // 使用应用上下文，避免Activity内存泄漏
35         this.mContext = context.getApplicationContext();
36         initTTS();
37     }
38
39     /**
40      * 初始化TTS语音引擎（异步）
41      */
42     private void initTTS() {
43         mTts = new TextToSpeech(mContext, status -> {
44             if (status == TextToSpeech.SUCCESS) {
45                 isInitSuccess = true;
46                 // 默认初始化中文播报
47                 setLanguage(Locale.CHINA);
48                 setSpeechParams(mSpeechRate, mPitch);
49                 setProgressListener();

```

```

50         if (mListener != null) {
51             mListener.onTtsInitSuccess();
52         }
53     } else {
54         isInitSuccess = false;
55         Log.e(TAG, "TTS引擎初始化失败, 设备无可用语音引擎");
56         if (mListener != null) {
57             mListener.onTtsInitFailed();
58         }
59     }
60 });
61 }
62
63 /**
64  * 设置播报进度全局监听
65  */
66 private void setProgressListener() {
67     mTts.setOnUtteranceProgressListener(new UtteranceProgressListener() {
68         @Override
69         public void onStart(String utteranceId) {
70             if (mListener != null) mListener.onSpeakStart(utteranceId);
71         }
72
73         @Override
74         public void onDone(String utteranceId) {
75             if (mListener != null) mListener.onSpeakDone(utteranceId);
76         }
77
78         @Override
79         public void onError(String utteranceId) {
80             if (mListener != null) mListener.onSpeakError(utteranceId);
81         }
82     });
83 }
84
85 /**
86  * 外部设置监听回调
87  */
88 public void setTTSListener(TTSListener listener) {
89     this.mListener = listener;
90 }
91
92 // ===== 语种切换工具方法 =====
93 public void setLanguage(Locale locale) {
94     if (!isInitSuccess || mTts == null) return;
95     int result = mTts.setLanguage(locale);

```

```

96         if (result == TextToSpeech.LANG_MISSING_DATA || result ==
TextToSpeech.LANG_NOT_SUPPORTED) {
97             Log.e(TAG, "当前设备不支持该语种，或缺失语音数据包");
98         }
99     }
100
101     public void setChinese() {
102         setLanguage(Locale.CHINA);
103     }
104
105     public void setEnglish() {
106         setLanguage(Locale.ENGLISH);
107     }
108
109     // ===== 语速音调自定义工具方法 =====
110     public void setSpeechParams(float rate, float pitch) {
111         if (!isInitSuccess || mTts == null) return;
112         mSpeechRate = rate;
113         mPitch = pitch;
114         mTts.setSpeechRate(rate);
115         mTts.setPitch(pitch);
116     }
117
118     public void setSlowSpeed() {
119         setSpeechParams(0.6f, 1.0f);
120     }
121
122     public void setFastSpeed() {
123         setSpeechParams(1.5f, 1.0f);
124     }
125
126     public void setGirlVoice() {
127         setSpeechParams(1.0f, 1.3f);
128     }
129
130     public void setManVoice() {
131         setSpeechParams(1.0f, 0.8f);
132     }
133
134     // ===== 核心播报方法 =====
135     /**
136      * 追加队列播报（不中断当前语音）
137      */
138     public void speak(String content) {
139         speak(content, TextToSpeech.QUEUE_ADD);
140     }
141

```

```

142     /**
143      * 立即播报 (打断当前所有语音, 清空队列)
144      */
145     public void speakImmediately(String content) {
146         speak(content, TextToSpeech.QUEUE_FLUSH);
147     }
148
149     /**
150      * 通用兼容播报方法
151      */
152     private void speak(String content, int queueMode) {
153         if (!isInitSuccess || mTts == null || content.isEmpty()) return;
154         String utteranceId = UUID.randomUUID().toString();
155         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
156             mTts.speak(content, queueMode, null, utteranceId);
157         } else {
158             HashMap<String, String> params = new HashMap<>();
159             params.put(TextToSpeech.Engine.KEY_PARAM_UTTERANCE_ID,
utteranceId);
160             mTts.speak(content, queueMode, params);
161         }
162     }
163
164     // ===== 播报控制方法 =====
165     public void stopSpeak() {
166         if (mTts != null) {
167             mTts.stop();
168         }
169     }
170
171     public boolean isSpeaking() {
172         return mTts != null && mTts.isSpeaking();
173     }
174
175     // ===== TTS引擎切换 =====
176     public Map<String, String> getInstalledTtsEngine() {
177         Map<String, String> engineMap = new HashMap<>();
178         if (mTts == null) return engineMap;
179         for (TextToSpeech.EngineInfo info : mTts.getEngines()) {
180             engineMap.put(info.name, info.label);
181         }
182         return engineMap;
183     }
184
185     public void switchTtsEngine(String enginePackageName) {
186         if (mTts != null) {
187             mTts.shutdown();

```



```

188     }
189     mTts = new TextToSpeech(mContext, status -> {
190         if (status == TextToSpeech.SUCCESS) {
191             isInitSuccess = true;
192             setLanguage(Locale.CHINA);
193             setSpeechParams(mSpeechRate, mPitch);
194         }
195     }, enginePackageName);
196 }
197
198 // ===== 文字转音频文件导出 =====
199 public void saveTextToAudio(String text, String filePath) {
200     if (!isInitSuccess || mTts == null) return;
201     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
202         mTts.synthesizeToFile(text, null, new java.io.File(filePath),
203             UUID.randomUUID().toString());
204     }
205
206 // ===== 资源释放 =====
207 public void release() {
208     if (mTts != null) {
209         mTts.stop();
210         mTts.shutdown();
211         mTts = null;
212     }
213     isInitSuccess = false;
214 }
215 }

```

五、标准调用示例

代码块

```

1 // 初始化TTS工具类
2 AdvancedTTS tts = new AdvancedTTS(this);
3 tts.setTTSListener(new AdvancedTTS.TTSListener() {
4     @Override
5     public void onTtsInitSuccess() {
6         // 初始化成功后再执行播报，彻底解决引擎未就绪报错
7         tts.speak("TTS初始化成功");
8     }
9
10    @Override
11    public void onTtsInitFailed() {
12        // 初始化失败处理：引导用户安装语音引擎/数据包

```

```

13     }
14
15     @Override
16     public void onSpeakStart(String id) {}
17
18     @Override
19     public void onSpeakDone(String id) {}
20
21     @Override
22     public void onSpeakError(String id) {}
23 });
24
25 // 自定义音色播报
26 tts.setGirlVoice();
27 tts.speak("萝莉音色播报测试");
28
29 // 强制打断播报
30 tts.speakImmediately("紧急播报内容");
31
32 // 页面销毁释放资源
33 @Override
34 protected void onDestroy() {
35     super.onDestroy();
36     tts.release();
37 }

```

六、高频报错 & 避坑总结

6.1 核心报错解决方案

报错：setLanguage failed: TTS engine connection not fully set up

原因：TTS异步初始化未完成，提前调用播报、语种设置API。

解决方案：所有TTS操作，必须在 `onTtsInitSuccess` 回调后执行，增加初始化状态判断。

6.2 开发必守规则

- 禁止在 `onCreate` 直接调用播报API，必须等待初始化成功
- 页面/组件销毁必须调用 `release()`，防止内存泄漏
- Android10+ 导出音频文件需适配分区存储，放弃传统文件路径
- 部分精简版系统缺失中文语音包，需引导用户在系统设置下载
- 重复初始化TTS需先销毁旧实例，避免多引擎冲突

七、版本适配总结

- **Android 5.0+**: 支持新版 `speak`、`synthesizeToFile` 重载方法
- **全版本**: 队列机制、语速音调、状态监听逻辑通用
- **高版本系统**: 限制后台服务常驻，必须主动调用shutdown释放引擎

(注：文档部分内容可能由 AI 生成)